

The big-DC MoE LLM design

I was asked, so here are some ideas I'd look at for a DC MoE LLM that I trained from scratch:

- 1) Stick with economical large systolic arrays, like you'll find in GPUs and TPUs. Don't buy expensive LPUs or other expensive low-batch designs.
- 2) Halve decode token latency by doing attention and FF in parallel instead of serially. Considering the ridiculous premium paid for fast token latency (what LPUs offer), a non-tiny model trained with serial att-then-FF is probably a silly model (at least in this respect).
- 3) Start sparse if hw can do it: train a double-sized model trained with 2x structured sparsity from the start. Maybe train dense for 1% of tokens and then redo with 50% lottery hypothesis structured sparsity. Evaluate this combined with or as an alternative to FP4.
- 4) Find the smallest FF layer that can run within-layer-parallelized on 64 of your chips with near 100% utilization. That's your expert size for MoE (or a multiple) - all experts will be 64 wide. You can store multiple experts on one 64x wide system if you like, but it will never process fewer weights than this size at a time in this design. You can of course do 32 width or 128 width or any other number - up to you. There are pros and cons to both larger and smaller. Going to 128x saves half the bandwidth requirement on the clos, but has a larger failure domain and a larger minimum expert size and wider dimensions in the torus.
- 5) Network topology is $N \times 64$. The second dimension (64) has enough bw to run a 64-wide expert internally without waiting on network with a 2-step pipeline - this requires high bandwidth but is cheap because it's local (can be a fast (expensive) router or a fast (cheap) 4x4x4 asymmetric-bw torus, an even-bw 3d 4x4x4 torus or even a 64-wide 1D torus, or an 8x8 torus, or could be a doubled-bw 8x8 mesh - you get the picture). The first dimension (N) consists of 64 parallel clos planes (one for each of the 64 nodes in the second dimension) that scale to any width you want, but with only 1/64th the bw per node to make clos economical per node - a clos is expensive per node, but not with only 1/64th the bandwidth. The MoE experts are all 64 wide, so to MoE this looks like a clos with N nodes each with 64x flops/storage/memory bw and also 64x the bw of what the clos actually has per node. So we run at 64x speed per node and pay for only 1x in the clos per node and without any MoE complications even if the 64 was e.g. a 4x4x4 asymmetric torus.
- 6) Have a few initial layers and a few final layers, with a single middle layer that has almost all the experts and that is repeated. So instead of e.g. 128 internal layers, there is one layer run 128 times with 128x as many experts. This makes the clos wider, which is OK - the point of a clos is that it can be wide. If you think this will make the LLM stupider at same flops/params, you are wrong, since you can simulate the standard layer structure perfectly inside this structure. It's just a question of how best to find the advantage here, not a question of whether it is there to be found.

7) Run the middle layer fewer times for cheap or easy tokens and more times for expensive or hard tokens. Can reduce kv cache significantly from this, but probably not the full 128x. RL for dynamic expert activation amount, expert choice, middle layer iteration count and attention KV cache usage seems promising. One option is to see all the prior layer KV caches with local attention, but only the final middle layer KV cache with global attention. The trouble is that seeing final layer activation with local attention requires the previous token to be done, defeating the parallelism inherent in prefill and training (and decode, with speculative decode). Global attention is what leads to huge KV caches, so this brings almost the full 128x reduction in KV cache storage and bandwidth requirements compared to the traditional global-everywhere transformer architecture. Global attention can see only the final vectors without serialization because far-away tokens is in a different batch and so have already been processed. Though this defeats parallelization across multiple batches for the same document - probably fine, but there are some design constraints to design around here. Super parallel prefill might simply operate in a different, less effective mode.

8) Consider experts with 2 or more layers internally to be able to run at half the clos bandwidth.

9) Support multicast natively in the network and place related (frequently co-activated) experts (for example multiples of the same one) near each other (on same bottom-most router in the clos). Then you can give the higher levels of the clos less bandwidth (deduplicated traffic costs less) and you can also reduce the up bw (more deduplicated) in the clos compared to the down bandwidth (less deduplicated). These factors combine and could be both quite significant if you activate many experts per layer.

10) Do not have two or more separate inference installations in the same DC. Combine them at the clos so that you get more even MoE expert usage. MoE is more efficient the more vectors there are at a time. With a large enough combined installation, you can get close to 100% utilization. This design is aiming at huge amounts of experts, so you are going to want all the simultaneous scale/batch that you can get to help make that efficient.

11) Do not run this system in lockstep - as soon as an expert or attention has enough tokens to do a step economically (or the wait has been too long), do it. So it is a continuous system. Fits well with one middle layer that is run a variable number of times.

12) Split the latent vector into two latent vectors per token. Both vectors are multiplied by the same weights, so this halves the number of parameters, with same flops. The second vector cannot be seen by attention and is only used to predict the next token. The first token can be seen as normal. This halves decode token latency and makes prefill take half as many flops (since it isn't trying to figure out the next token, which it doesn't need to predict anyway). The top few layers are then decode-only and they can also be omitted from prefill.

13) For decode, do speculative token prediction 4x (or more). Please, do not bolt this on later - train for this from the start of training, but only read it off the next-token-specific second latent

vector, not the general first one, so that these calculations are not done during prefill. A model trained without speculative token prediction from the start these days is a silly, silly model!

14) Do not disaggregate prefill and decode. Instead use managed aggregation - each unit receives a managed ratio of prefill and decode so that its flops and memory bandwidth are used evenly. This gives the benefits of disaggregation without the downsides, revealing disaggregation to be a silly idea. Unless you use an entirely separate model, distilled only for prefill, that is an encoder and not a decoder, then MAYBE it makes sense. But nobody is doing that, so it's silly as-is.

15) Use indexed attention. Not just somewhat sparse attention (already popular), actually indexed attention (not popular), like people do for RAG, but for attention. Something that is $\sim O(\log n)$, not $O(n)$. If you do it right, you can stick the ancient part of the KV cache on an SSD or even farm it out to a storage cluster and have that be enough bw (!). No one has made this work at scale and reported about it. I suggest you do. Google doesn't look at every page on the internet for each search query, it uses an index. That's what you should do for attention, too, and for the same reasons.

16) Don't buy much HBM per chip. 4 GB should do it. You don't need much HBM for kv cache since you offloaded it via indexed attention and your experts have as few weights as possible to keep a 64x unit busy. You don't need 100s of GBs of HBM per chip. 4 GB per node lets a 64x wide unit have 256 GB of HBM. If you have many nodes, this is more than enough even for gigantic models. You'll find that no one even sells HBM in units so small as 4 GB. Consider using not using HBM at all for this reason, preferring ganging up multiples of other kinds of RAM that give similar bandwidth combined. You heard it here first: the AI industry has created a global run on DRAM capacity for no reason at all. We would never have needed all that HBM capacity if we had done a better job as AI researchers and engineers.

17) Extract as much knowledge of the world into a vector database that you retrieve from at every iteration of the middle layer. My preferred method is a giant text -> detailed entailments -> vectors pipeline, but you can use any pipeline you want. Not same as RAG, since this is inside every layer and only retrieves 1 vector (or a few), but still somewhat similar. This is almost the same as placing all knowledge of the world into the attention span, since we will use indexes to pull out relevant information from both, but this is available during training so that you don't waste weights on knowledge that can be retrieved from a database. The database has to be correct, but not complete - you are encouraging the model to get information from the database during and after training, not making it completely unable to remember things in its weights.

18) In addition to experts, allow people to fine-tune tiny QLoRAs for specialized LLMs. So tiny that you can retrieve a QLoRA for every vector and apply it to just that vector without extreme inefficiency (maybe don't even multiply all the latent entries onto it, to keep it small). As well as letting people expand the within-layer vector database for their bespoke models, of course. This allows a giant multi-billion-dollar cloc LLM to serve many specialized models simultaneously from a 99.99% shared spine at near 100% efficiency even if each specialized model only

receives a tiny bit of traffic. For bespoke uses with large amounts of traffic, it instead becomes economical to sell people their own bespoke experts within the larger system - again using the cheap scaling properties of the MoE clos. This should be part of any serious large AI infrastructure business offering, though as far as I know nobody does this.

19) Don't distribute heavy LLM inference across many DCs, only lightweight edge LLMs need to go everywhere. An extra 50 ms or even 100 ms of latency on serious AI business is inconsequential - people are already waiting seconds, minutes even hours on LLM responses. The bigger your installation, the more experts you can use and the less the cost of MoE will be. It's free intelligence for your AI. We might want a future with a single big AI DC where all the world's heavy AI work happens in that one location. Preferably in Greenland, north Canada, Iceland or perhaps north Norway for better cooling and nuclear power generation economics (not that I think this will happen). Or maybe in orbit, where solar is 4x as efficient and can be baseload (I'm skeptical about this in the near term, but eventually it will make sense).

These are not 19 independent design points, it is a cohesive whole, though pieces of it can be done without the rest.

I also wrote an in-depth post about how to create AI hardware and software here:
<https://lnkd.in/p/gNhBPiVj>

Is all of this a good idea? Well I certainly never built this system, so I can't prove any of it that way, but I think these are interesting ideas that may well work great together and I haven't seen this all put together in this way. Even if you don't do any of this, if you work in this area, might be a good idea to be sure that you have a good reason for not doing each of these things and for not combining things in this or a similar way. I know I'd be going through this list point by point if I were still being paid to work on this and someone posted such a list. If even one of these things work out, and you wouldn't have done it otherwise, that could be your promotion.